

Lecture 1

Definition

Cloud Computing is using **remote servers over the internet** to store, manage, and process data, instead of local computers.

It provides **on-demand access** to computing resources like storage, software, and networks.

Advantages of Cloud Computing

- **Scalability:** Add or remove resources easily.
- **Cost Efficiency:** Pay only for what you use (no need for expensive hardware).
- **Accessibility:** Access from anywhere via the internet.
- **Automatic Updates:** Providers handle upgrades and maintenance.
- **Backup and Recovery:** Easier data recovery in case of system failure.

Disadvantages of Cloud Computing

- **Data Security and Privacy Risks:** Sensitive data is stored online.
 - **Vendor Lock-In:** Difficult to move to another provider.
 - **Hidden Costs:** Long-term usage can increase costs.
 - **Downtime and Reliability Issues:** Service may go down.
 - **Limited Customization:** You can't always change or modify the service.
-

Components of the Cloud

1. **Front-End:** The user interface (desktop, mobile, thin clients).
 2. **Back-End:** Servers that handle processing and data management.
 3. **Storage / Datacenters:** Where data is saved.
 4. **Delivery Services:**
 - **SaaS (Software as a Service)** – apps like Gmail, Zoom.
 - **PaaS (Platform as a Service)** – platforms for developers (like Google App Engine).
 - **IaaS (Infrastructure as a Service)** – virtual machines and servers (like AWS EC2).
-

Cloud Computing Stack

- **Applications:** The software users interact with.
- **Development Platforms:** Tools for building and deploying apps.
- **Elasticity:** Scaling automatically based on demand (uses APIs, triggers, alarms).
- **Sharing Mechanisms:** Virtualization and containers.

- **Distributed Systems:** Manage storage and resources across many computers.
 - **Data Centers:** Physical infrastructure behind cloud services.
-

Delivery Models (Building Blocks)

These are the **core types** of cloud service:

- **SaaS:** Software delivered via the web.
 - **PaaS:** Platform for developers to build apps.
 - **IaaS:** Infrastructure (VMs, storage, networking).
-

Back to Basics — Parallel Computing

- Performs **many calculations at once** to save time.
 - Helps solve large problems faster.
 - **Amdahl's Law:**
 - Measures how much speedup is possible.
 - Formula: $S = T(1) / T(N)$
 - If only part of the program is parallel, total speedup is limited.
-

Back to Basics — Distributed Systems

- A **network of autonomous computers** working together for a common goal.
 - Users see it as **one single system**.
 - **Characteristics:**
 1. Appears as one system.
 2. Each computer is independent.
 3. Each has its own security and resource control.
 4. Multiple failure points.
 5. Resources may not always be available.
 6. Can scale easily.
 7. Can stay available even with some failures.
-

Performance Metrics

- **Throughput:** Data processed per second (system view).
- **Latency:** Response time (user view).
- **Utilization:** How much system capacity is used (future view).

- If utilization > 60%, performance drops.
 - Average server utilization: 15–20% in old data centers.
 - Use same units: MB/s, GB/s (1B = 8 bits).
-

Datacenter Hardware

- Houses **50K–200K servers**, consuming **10–100MW** of power.
 - **Compute:** Multi-core CPUs, GPUs, AI accelerators (TPU, FPGA).
 - **Storage:** HDD, SSD, NVMe, NVM Flash, JBOD arrays.
 - **Networking:** 40–200 GbE NICs and switches, Smart NICs, SDN, in-network processing.
-

Total Cost of Ownership (TCO)

TCO = CapEx (capital) + OpEx (operational)

- **Operators view:**
 - *CapEx*: buildings, power, servers, cooling.
 - *OpEx*: electricity, staff, maintenance.
 - **Users view:**
 - *CapEx*: hardware leases.
 - *OpEx*: pay-per-use cloud services.
 - Goal: **maximize hardware utilization** to reduce TCO.
-

Reliability and Availability

- **FIT (Failure in Time):** Failures per billion hours.
- **MTTF (Mean Time To Failure):** Time before the first error.
- **MTTR (Mean Time To Repair):** Time to fix a failure.
- **Availability = $MTTF / (MTTF + MTTR)$**

Techniques to increase availability:

- Replication
- Sharding (partitioning)
- Load balancing
- Watchdog timers
- Integrity checks
- Eventual consistency
- Graceful degradation (limited service instead of full failure)

VMware Stack

- **RAID:** Combines disks for redundancy.
- **SAN:** Storage Area Network.
- **NAS:** Network Attached Storage.

Containers and Kubernetes

- **Containerization:** Packages apps with all dependencies.
- **Kubernetes:** Manages clusters of containers.
- **Multi-cloud Deployment:** Apps can run across Azure, GCP, etc.
- Example:
 - **Frontend containers** (WeChat, Uber, etc.)
 - **Backend containers** managed by **AKS (Azure Kubernetes Service)**
 - **Load balancers** distribute traffic.

Lecture 2

1. Introduction to Computer Virtualization

- **Definition:**
Computer Virtualization is the process of **creating virtual versions** of physical resources such as **servers, storage, or networks**, by **separating hardware from software**.
(يعني فصل الهاردوير الحقيقي عن النظام التشغيلي وتشغيل نسخ افتراضية منه) →
- It allows **multiple virtual machines (VMs)** to run on one **physical machine**, maximizing resource use.
- **Relation to Cloud Computing:**
Virtualization is the **foundation** of Cloud Computing.
It mainly provides the **IaaS (Infrastructure as a Service)** layer, while Cloud adds **PaaS and SaaS**.

2. Core Components of Virtualization

Component	Description	Arabic Explanation
Host	The actual physical hardware.	الجهاز الحقيقي
Hypervisor / VMM	Software that creates and manages VMs.	مدير الأنظمة الوهمية
Guest	The virtual machine (with its OS) running on the host.	النظام الوهمي

VM (Virtual Machine) A virtual computer with its own CPU, RAM, storage, and network. كمبيوتر وهمي متكامل

Each VM acts **independent**, like a separate computer.

3. Hypervisors (Virtual Machine Monitors - VMMs)

◆ Type 0 – Hardware/Firmware-based

- Built directly into hardware (firmware).
- Examples: **IBM LPARs, Oracle LDOMs.**
- (موجود في مستوى الهاردوير نفسه مش برنامج منفصل)

◆ Type 1 – Bare-Metal Hypervisor

- Runs **directly on the hardware** (no host OS).
- Efficient, secure, and used in **data centers**.
- Examples: **KVM, VMware ESXi, Microsoft Hyper-V.**
- (الأكثر استخداماً في السيرفرات لأنه مباشر وسريع)

◆ Type 2 – Hosted Hypervisor

- Runs **as a program** on top of a normal operating system.
- Used in **desktop and testing** environments.
- Examples: **VMware Workstation, Oracle VirtualBox, Parallels Desktop.**
- (برنامج بتشغله على نظامنا لتجربة أنظمة أخرى)

4. Key Characteristics of Virtualization

Characteristic	Meaning	Arabic Hint
Partitioned	Divides physical resources (CPU, RAM) among VMs.	تقسيم الموارد
Isolated	Each VM runs separately for security and stability.	مستقل VM كل
Encapsulated	VM stored as a group of files (config, disk, BIOS). Enables easy migration.	ملف واحد سهل النقل
Hardware Independent	VM can run on any compatible hardware.	غير مرتبط بجهاز معين
Load Balancing	Distributes workloads to prevent overload.	توزيع الأحمال
Performance Tuning	Adjusting resources for better performance.	تحسين الأداء

5. Compute Virtualization

Virtualization involves **CPU, memory, and I/O.**

✚ CPU Virtualization

- CPUs use **protection rings (0–3)** to control privileges:

- **Ring 0:** highest privilege (OS, drivers).
 - **Ring 3:** lowest privilege (user apps).
→ (الحلقات بتحدد مين له صلاحية الوصول للهاردوير)
 - **Privileged instructions** (e.g., hardware control) are allowed only in Ring 0.
 - In a VM, the VMM **intercepts** these instructions to protect the system.
-

6. Building Blocks of Virtualization

Virtual CPU (VCPU)

- VMM creates a **virtual CPU** per VM that represents the CPU's state.
 - When the VM runs, VMM loads this state; when paused, it saves it.
-

Trap and Emulate

- The CPU normally has two modes: **user** and **kernel**.
- In virtualization, both guest modes run in **real user mode** for safety.
- When a guest OS tries a **privileged instruction**, it **traps** to the VMM.
- The VMM **emulates** the instruction and returns control.
→ (يعني الهايبرفايسر ببحاكي الأمر بدل ما النظام ينفذه بنفسه)

✅ User-mode code runs fast, but kernel-mode is slower due to this trapping.

💡 Modern CPUs add virtualization hardware to reduce this cost.

Nested Page Tables (NPT)

- Used for **memory management** in VMs.
 - Each guest manages its own **virtual** → **physical** address map.
 - VMM keeps another mapping (**nested tables**) to the real memory.
→ (جدول ترجمة إضافي يحتفظ به الهايبرفايسر لكل ضيف)
 - This adds overhead and can slow performance (more TLB misses).
-

7. Types of Virtual Machines

Application Containment

- Runs applications in **isolated zones/containers** within one OS.
- Example: **Oracle Zones / Containers**.
- Each zone has its own network, users, and resources.
- Only one **kernel** (the host OS) runs.
→ (بيشغل تطبيقات مستقلة داخل نفس النظام – Docker containers زي)

8. Virtualization and Operating System Components

A. CPU Scheduling

- Even one CPU system acts like multiprocessor when virtualized.
- Each VM has one or more **VCPUs**.
- If CPUs < VMs → **over-commitment** occurs.
- VMM uses **scheduling algorithms** (with fairness) to assign CPU time.
→ (يقسم وقت المعالج بين الأنظمة الوهمية بعدل)

B. Live Migration

- Move a running VM from one host to another **without interruption**.
→ (نقل النظام الوهمي أثناء التشغيل بدون توقف)

Steps:

1. Source connects to target VMM.
2. Target creates new VM shell.
3. Source sends read-only pages.
4. Sends read-write pages (cleaned).
5. Repeats until few dirty pages left.
6. Freezes VM, sends final state, starts VM on target.
7. Source shuts down its copy.

Used for: maintenance, load balancing, zero downtime.

9. Examples

VMware

- **Type 2 hypervisor** running on x86 (Windows/Linux).
- Can host multiple OSs (Windows, Linux, etc.).
- Provides each guest with virtual CPU, RAM, disks, network.
- Disks can be real or **virtual files** inside the host filesystem.
→ (يعني كل نظام وهمي لديه ملفات تمثل الهارد بتاعه)

Java Virtual Machine (JVM)

- Example of **software-level (programming)** virtualization.
- Created by **Sun Microsystems (1995)**.
- Idea: **“Write Once, Run Anywhere.”**

Process:

1. Java code → compiled to **bytecode (.class)**.
2. **JVM** loads and executes the bytecode.
3. Includes **Garbage Collection** to free unused memory.
4. Uses **JIT compiler** to speed up execution by turning bytecode into native code.
→ (مفسرة ومترجمة في نفس الوقت Java يعني)

Lecture 3

VMM (Virtual Machine Monitor)

- **Role:** Runs multiple Virtual Machines (VMs) on one Physical Machine (PM).
→ (يدير أكثر من جهاز افتراضي على جهاز حقيقي واحد)
- **Similar to OS:** Like how OS runs multiple processes on CPU.
- **Performs machine switch:** Runs one VM, saves context, switches to another.
- **Problem:** Guest OS expects full hardware access, but must be isolated.

Trap and Emulate

- **CPU privilege levels (rings):**
 - Ring 0 = OS (highest privilege)
 - Ring 3 = User apps (lowest)
- **With VMM:**
 - VMM runs in Ring 0
 - Guest OS runs in Ring 1
 - Guest apps run in Ring 3
- **Trap-and-emulate:** When Guest OS executes privileged instruction, it traps to VMM which emulates it.
→ (VMM أي أمر حساس يتم التقاطه وتنفيذه بواسطة الـ)

Trap and Emulate Process

1. Guest app triggers syscall/interrupt → trap to VMM.
2. VMM redirects trap to Guest OS handler.
3. Guest OS handles it, but privileged instructions again trap to VMM.
4. VMM emulates privileged action (e.g., set IDT, set CR3, hardware access).
→ (VMM أي عملية تحتاج صلاحيات عالية تمر عبر الـ)

Problems with Trap and Emulate

- Guest OS can detect it's not in Ring 0 (by checking CPU registers).
 - Some **sensitive instructions** don't trap but behave differently in Ring 1.
 - Example: **popf** instruction → can't set interrupt flag (IF) in Ring 1.
 - **Reason:** x86 architecture wasn't designed for virtualization.
→ (بعض التعليمات لا تلتقط، فتسبب أخطاء في النظام الضيف)
-

Popek-Goldberg Theorem

- **Sensitive instruction:** Changes hardware state.
 - **Privileged instruction:** Only runs in privileged mode.
 - For trap-and-emulate to work:
Sensitive $\subseteq$ Privileged.
 - **x86 fails this rule**, so pure trap-and-emulate isn't possible.
→ (لا يدعم الافتراضية الكاملة بهذه الطريقة x86)
-

x86 Virtualization Techniques

1. Paravirtualization

- Modify Guest OS to make “hypercalls” to VMM.S
- Needs source code changes.
- **Example:** Xen.
→ (يتطلب تعديل نظام التشغيل نفسه)

2. Full Virtualization

- No OS modification needed.
- Uses dynamic binary translation to handle sensitive instructions.
- Slower due to translation overhead.
- **Example:** VMware.
→ (يترجم التعليمات أثناء التشغيل)

3. Hardware-Assisted Virtualization

- Supported by modern CPUs (Intel VT-x, AMD-V).
 - CPU has **VMX mode** to run Guest OS safely in Ring 0.
 - **Example:** KVM / QEMU.
→ (المعالج يساعد في الافتراضية من خلال أوضاع خاصة)
-

Memory Virtualization

- **Address levels:**
 - GVA → GPA (by Guest OS)

- GPA → HPA (by VMM/Host OS)
→ (كل نظام ضيف يظن أن لديه ذاكرة خاصة به)

Techniques:

1. Shadow Paging:

- VMM creates a combined GVA → HPA mapping.
- Tracks and updates when Guest page table changes.
→ (VMM جدول صفحات ظلي يُدار من الـ)

2. Extended Page Tables (EPT):

- Hardware supports two levels of translation.
- Faster but needs hardware support.
→ (ترجمة مزدوجة مدعومة من العتاد)

I/O Virtualization

- Guest OS needs access to I/O devices but can't control them fully.
- **Two techniques:**
 1. **Emulation:** Guest I/O traps to VMM, executed by host.
 2. **Device Passthrough:** Assign part of device directly to VM.
→ (محاكاة الأجهزة أو تمرير مباشر للجهاز)